Adobe PS: Selection Assignment

In one day, build a source-traced knowledge base and a reproducible data pipeline for one programme at our institute, stopping right before the embedding step.

ground rules

- **Time window:** 24 hours from release.
- **Individual work:** each candidate submits their own repo or zip
- **AI tools are allowed, with disclosure.**
- **Data ethics:** use only publicly available institute information and openly licensed content (NPTEL / SWAYAM, MIT OpenCourseWare and similar). Record the licence for every external resource. No real student records, grades or personal details.
- **No embeddings yet.** Do not call any embedding or LLM API. Your pipeline ends with clean, chunked, metadata-rich files that an embedding step could consume on day two. Python is preferred, but any stack is fine if it runs from one command.

Your task

Pick **one programme** at our institute (for example B.Tech in one branch) and cover it in depth rather than the whole institute thinly.

Part A: Collect the data

Gather these from official, public sources. Every record must carry a source URL and the date you retrieved it.

1. **Course catalogue:** at least 15 courses with code, title, department, credits, semester offered, course type (core / elective / open), mode, syllabus or learning objectives, and prerequisites.
2. **Academic rules:** credit limits per semester, minor eligibility (CGPA or other criteria), prerequisite rules, and the policy on earning credit through NPTEL / SWAYAM if the institute allows it. Store each rule as a separate, quotable entry rather than one blob.
3. **Open learning resources:** at least 30 resources (NPTEL, SWAYAM, MIT OCW, other open educational resources) mapped to the courses or skills they support, with licence, format, and rough duration.

Sources may be PDFs, web pages or spreadsheets, so expect messy input. Scraping is fine if it is polite (respect robots.txt and rate limits) and legal; manual entry is fine for small, hard-to-parse pieces, but say so in the README.

Part B: Model the data

Design a schema that lets a program reason about prerequisites, eligibility and skill coverage. The table below is a starting point, not a requirement. Justify any change in the README.

Part C: Build the pipeline(hld , run on small sample)

One command (for example `python run_pipeline.py` or `make pipeline`) must take raw files to final outputs, with these stages:

1. **Ingest:** raw files saved untouched in `data/raw/`.
2. **Parse and normalise:** consistent codes, credit numbers, semester labels and names across sources.
3. **Validate:** fail loudly on problems such as a prerequisite that names a missing course, a prerequisite cycle, non-numeric credits, a duplicate course code, or a record with no source URL.
4. **Chunk:** split syllabi, rules and resource descriptions into retrieval-sized chunks. Each chunk keeps its parent ID, type, source and any fields needed for filtering.
5. **Export:** write final CSV / JSON files to `data/processed/` (the future seed data) and a chunks file such as `chunks.jsonl`, ready for embedding.

You need to design it what would you expect from it each step , you can run the pipeline on small sample dataset not full.

Part D: Data quality report (1 page)

Report how many courses, rules, resources and chunks you collected; what percentage of fields are filled per entity; which checks your validator runs and what it caught; what is missing or uncertain; and how a teammate could verify a sample of records against sources.

Part E: Ideas note (1 page)

Tell us how you would use this data. Answer briefly:

- Which three student questions could the buddy answer well with this data, and which could it not?

Starting schema

Every entity also needs `source_url`, `retrieved_on` and `source_type` (official / open-licence / manual).

Entity	Key fields	Why the buddy needs it
Course	code, title, dept, credits, semester offered, type, mode, objectives	Plan building and citing real course codes
Prerequisite edge	course, requires, kind (hard / recommended)	Ordering and eligibility checks
Skill	id, name, level, parent skill	Gap analysis against a goal
Course-skill link	course, skill, depth (intro / working / advanced)	Which course teaches what
Academic rule	id, category, text, numeric limit, applies to	Constraint checking with quotable rules

Entity	Key fields	Why the buddy needs it
Resource	id, title, provider, licence, format, duration, URL	Open content to fill gaps
Resource-skill link	resource, skill, level	Choosing content for a gap
Module / lesson	id, parent course or resource, order, summary	Fine-grained recommendations
Exercise (bonus)	id, skill, difficulty, question, answer key	Practice and mastery checks
Chunk	chunk id, parent id, type, text, filter metadata	Input for the future embedding step

Deliverables

Submit one Git repository (GitHub link, or a zip if you prefer) containing:

- ☐`data/raw/` with the untouched source files, and `data/processed/` with final CSV / JSON files
- ☐`chunks.jsonl` (or equivalent) with parent IDs, types, sources and filter metadata
- ☐ `SOURCES.md` listing every source with URL, retrieval date and licence
- ☐`IDEAS.md` (Part E, one page)

Submission and next steps

- **Deadline:** 24 hours after you receive this brief (exact time in the message that sent it).
- **Submit:** reply with the repo link or zip. Late submissions are judged on what was in by the deadline.
- **Questions:** ask for clarifications any time. Good clarifying questions count in your favour; guessing silently does not.